

**A Machine Learning-based Intrusion detection for detecting Internet of Things
Network Attacks**

Iswarya P

B.Tech Computer Science and Engineering (IoT & Automation)

May 2025

Abstract

Due to rapid growth in IoT, there has been an immense improvement in connectivity as well as in automation. However, serious security issues have emerged in this process. IoT devices with constrained resources and scalability suffer highly sophisticated cyberattacks. Traditional security measures usually fail to address the issues in IoT challenges. The present study addresses this challenge by exploring machine learning-based approaches for detecting and mitigating IoT network attacks. The methodology begins with min-max normalization using the UNSW-NB15 dataset, which captures a variety of modern attack types, in order to standardize the data and reduce information leakage. The complexity of the dataset is reduced while preserving the essential features using Principal Component Analysis (PCA). Six machine learning models, including XGBoost, CatBoost, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Quadratic Discriminant Analysis (QDA), and Naïve Bayes (NB), are used for normal and malicious traffic classification. In evaluating the system's performance, one uses precision, recall, F1-score, and Matthew Correlation Coefficient (MCC). This methodology provides well organized and scalable method for enhancing IoT networks, addressing unique issues in a resource-constrained environment

KEY WORDS: IDS – Intrusion Detection System, PCA – Principal Component Analysis, SVM – Support Vector Machine

CHAPTER 1

SUMMARY OF THE BASE PAPER

Title	: A Machine Learning-based intrusion detection for detecting internet of things Network attacks
Publisher	: IEEE
Indexed in	: SCIE
Year	: 2022
DOI	: 10.1016/j.aej.2022.02.063
Base paper URL	: https://doi.org/10.1016/j.aej.2022.02.063

The base paper emphasis on improving IoT security network attacks by using Machine Learning techniques to develop an Intrusion Detection System (IDS). This paper uses UNSW-NB15 dataset that captures mixed of contemporary attacks and normal activities of network traffic grouped into nine different attack types suitable for simulating real-world IoT network scenarios. This dataset initially undergoes preprocessing by using Min-Max Normalization to have all the feature values on the same scale and compute Principal Component Analysis (PCA) for dimensionality reduction.

The processed dataset is used to train and test six different supervised ML algorithms like XGBoost, CatBoost, k- Nearest Neighbors (KNN), Support Vector Machine (SVM), Quadratic Discriminant Analysis (QDA), and Naïve Bayes (NB).

The model evaluates all model's performance using performance metrics such as Accuracy, Precision, Area Under Curve (AUC) , Recall, F1-Score, Kappa, Matthews Correlation Coefficient (MCC).

Among the models, XGBoost and CatBoost achieved the maximum accuracy.

CHAPTER 2

MERITS AND DEMERITS OF THE BASE PAPER

Merits

1. The paper uses the latest UNSW-NB15 dataset that captures various recent network attacks that makes the model highly relevant and practical for present day IoT network scenarios.
2. It uses min–max normalization which ensures feature scaling, while Principal Component Analysis (PCA) efficiently reduces dimensionality, improving overall computational efficiency.
3. Model evaluation using six different machine learning models for better comparison.

Demerits

1. The paper lacks real-world testing such as deployment on actual IoT devices or real time IoT infrastructure.
2. The paper focuses majorly on metrics like Accuracy, AUC, Precision, Recall, F1, MCC but does not consider other metrics like latency, throughput, and memory usage

CHAPTER 3

SOURCE CODE

3.1 EXPLORATORY DATA ANALYSIS

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import numpy as np

from scipy import stats

df = pd.read_csv(r"C:\Users\ISWARYA P\Downloads\UNSW_NB15_training-set.csv")

print("Dataset Shape:", df.shape)

print("\nColumn Names:\n", df.columns)

print("\nData Types and Missing Values:\n")

print(df.info())

print("\nFirst Five Rows:\n", df.head())

print("\nMissing Values Count:\n", df.isnull().sum())

print("\nSummary Statistics:\n", df.describe())

categorical_cols = ['attack_cat', 'proto', 'service', 'state']

for col in categorical_cols:

    print(f"\nUnique values in {col}:\n", df[col].value_counts())

    print("\nLabel Distribution:\n", df['label'].value_counts())

    print("\nAttack Category Distribution:\n", df['attack_cat'].value_counts())
```

```

plt.figure(figsize=(12,6))

sns.histplot(df['dur'], bins=50, kde=True)

plt.title('Distribution of Duration')

plt.show()

plt.figure(figsize=(12, 12))

sns.heatmap(df.select_dtypes(include=['number']).corr(), annot=False, cmap='coolwarm')

plt.show()

z_scores = np.abs(stats.zscore(df.select_dtypes(include=['float64', 'int64'])))

outliers = (z_scores > 3).sum(axis=0)

print("\nOutlier Count per Feature:\n", outliers)

```

3.2 DATA PREPROCESSING

3.2.1 MIN-MAX NORMALIZATION

```

import pandas as pd

from sklearn.preprocessing import MinMaxScaler, LabelEncoder

dataset = pd.read_csv(r"C:\Users\ISWARYA P\Downloads\UNSW_NB15_training-set.csv")

numerical_cols = dataset.select_dtypes(include=['int64', 'float64']).columns

numerical_cols = [col for col in numerical_cols if col not in ['id', 'label']]

scaler = MinMaxScaler()

dataset[numerical_cols] = scaler.fit_transform(dataset[numerical_cols])

label_encoder = LabelEncoder()

```

```

dataset['attack_cat'] = label_encoder.fit_transform(dataset['attack_cat'])

dataset.to_csv('processed_dataset.csv', index=False)

print('Class mappings:', dict(zip(label_encoder.classes_, range(len(label_encoder.classes_)))))

print("Dataset preprocessing complete! Saved as 'processed_dataset.csv'")

```

3.2.2 PCA

```

import pandas as pd

from sklearn.decomposition import PCA

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import classification_report, accuracy_score

import matplotlib.pyplot as plt

data = pd.read_csv('processed_dataset.csv')

numeric_data = data.select_dtypes(include=['int64', 'float64'])

pca = PCA(n_components=10)

principal_components = pca.fit_transform(numeric_data)

pca_df = pd.DataFrame(data=principal_components, columns=[f'PC{i+1}' for i in range(10)])

pca_df['attack_cat'] = data['attack_cat']

pca_df.to_csv('pca_transformed_dataset.csv', index=False)

X = pca_df.drop('attack_cat', axis=1)

y = pca_df['attack_cat']

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Classification Report:\n", classification_report(y_test, y_pred))

print("Accuracy:", accuracy_score(y_test, y_pred))

plt.figure(figsize=(8, 5))

plt.plot(range(1, 11), pca.explained_variance_ratio_.cumsum(), marker='o', color='b', label='Cumulative
Explained Variance')

plt.axhline(y=pca.explained_variance_ratio_.cumsum()[-1], color='r', linestyle='--', label=f'Total Retained:
{pca.explained_variance_ratio_.cumsum()[-1]*100:.2f}%')

plt.title('Cumulative Explained Variance (10 Components)')

plt.xlabel('Number of Principal Components')

plt.ylabel('Cumulative Explained Variance')

plt.legend()

plt.grid(True)

plt.show()

import pandas as pd

import numpy as np

import time

import xgboost as xgb

from sklearn.model_selection import train_test_split

```



```

from sklearn.metrics import (
    accuracy_score, roc_auc_score, recall_score, precision_score,
    f1_score, cohen_kappa_score, matthews_corrcoef )

from sklearn.preprocessing import label_binarize

import matplotlib.pyplot as plt

import seaborn as sns

df = pd.read_csv("pca_transformed_dataset.csv")

X = df.iloc[:, :-1] # Features (PCA components)

y = df.iloc[:, -1] # Target (multi-class labels)

classes = np.unique(y)

num_classes = len(classes)

print(f"Number of unique classes: {num_classes}") # Should be 10

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)

xgb_model = xgb.XGBClassifier(
    objective="multi:softprob",
    eval_metric="mlogloss",
    use_label_encoder=False,
    num_class=num_classes,
    n_estimators=100,
    learning_rate=0.1,
    max_depth=6,

```

```

subsample=0.8,

    colsample_bytree=0.8)

start_time = time.time()

xgb_model.fit(X_train, y_train)

training_time = time.time() - start_time

y_pred = xgb_model.predict(X_test)

y_pred_proba = xgb_model.predict_proba(X_test) # Probability predictions for all classes

y_test_bin = label_binarize(y_test, classes=classes)

accuracy = accuracy_score(y_test, y_pred) * 100

auc = roc_auc_score(y_test_bin, y_pred_proba, average="macro") * 100 # Multi-class AUC

recall = recall_score(y_test, y_pred, average="macro") * 100

precision = precision_score(y_test, y_pred, average="macro") * 100

f1 = f1_score(y_test, y_pred, average="macro") * 100

kappa = cohen_kappa_score(y_test, y_pred) * 100

mcc = matthews_corrcoef(y_test, y_pred) * 100

print(f'Accuracy: {accuracy:.2f}%')

print(f'AUC: {auc:.2f}%')

print(f'Recall: {recall:.2f}%')

print(f'Precision: {precision:.2f}%')

print(f'F1 Score: {f1:.2f}%')

print(f'Kappa: {kappa:.2f}%')

```

```

print(f"MCC: {mcc:.2f}%")

print(f"Training Time: {training_time:.4f} seconds")

feature_importance = xgb_model.feature_importances_

feature_names = X.columns

importance_df = pd.DataFrame({"Feature": feature_names, "Importance": feature_importance})

importance_df = importance_df.sort_values(by="Importance", ascending=False)

plt.figure(figsize=(10, 5))

sns.barplot(x="Importance", y="Feature", data=importance_df, palette="Blues_r")

plt.title("Feature Importance Plot (XGBoost)")

plt.xlabel("Variable Importance")

plt.ylabel("Features")

plt.show()

```

3.3 MODEL TRAINING

3.3.1 XGBOOST

```

import pandas as pd

import numpy as np

import time

import xgboost as xgb

from sklearn.model_selection import train_test_split

from sklearn.metrics import (

```

```

accuracy_score, roc_auc_score, recall_score, precision_score,

f1_score, cohen_kappa_score, matthews_corrcoef)

from sklearn.preprocessing import label_binarize

import matplotlib.pyplot as plt

import seaborn as sns

df = pd.read_csv("pca_transformed_dataset.csv")

X = df.iloc[:, :-1] # Features (PCA components)

y = df.iloc[:, -1] # Target (multi-class labels)

classes = np.unique(y)

num_classes = len(classes)

print(f'Number of unique classes: {num_classes}')

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)

xgb_model = xgb.XGBClassifier(

    objective="multi:softprob", # Multi-class classification

    eval_metric="mlogloss", # Log-loss for multi-class problems

    use_label_encoder=False,

    num_class=num_classes, # Set number of classes

    n_estimators=100,

    learning_rate=0.1,

    max_depth=6,

    subsample=0.8,

```

```

colsample_bytree=0.8)

start_time = time.time()

xgb_model.fit(X_train, y_train)

training_time = time.time() - start_time

y_pred = xgb_model.predict(X_test)

y_pred_proba = xgb_model.predict_proba(X_test) # Probability predictions for all classes

y_test_bin = label_binarize(y_test, classes=classes)

accuracy = accuracy_score(y_test, y_pred) * 100

auc = roc_auc_score(y_test_bin, y_pred_proba, average="macro") * 100 # Multi-class AUC

recall = recall_score(y_test, y_pred, average="macro") * 100

precision = precision_score(y_test, y_pred, average="macro") * 100

f1 = f1_score(y_test, y_pred, average="macro") * 100

kappa = cohen_kappa_score(y_test, y_pred) * 100

mcc = matthews_corrcoef(y_test, y_pred) * 100

print(f"Accuracy: {accuracy:.2f}%")

print(f"AUC: {auc:.2f}%")

print(f"Recall: {recall:.2f}%")

print(f"Precision: {precision:.2f}%")

print(f"F1 Score: {f1:.2f}%")

print(f"Kappa: {kappa:.2f}%")

print(f"MCC: {mcc:.2f}%")

```

```

print(f"Training Time: {training_time:.4f} seconds")

feature_importance = xgb_model.feature_importances_

feature_names = X.columns

importance_df = pd.DataFrame({"Feature": feature_names, "Importance": feature_importance})

importance_df = importance_df.sort_values(by="Importance", ascending=False)

plt.figure(figsize=(10, 5))

sns.barplot(x="Importance", y="Feature", data=importance_df, palette="Blues_r")

plt.title("Feature Importance Plot (XGBoost)")

plt.xlabel("Variable Importance")

plt.ylabel("Features")

plt.show()

```

3.3.2 CATBOOST

```

import pandas as pd

import numpy as np

import time

import seaborn as sns

import matplotlib.pyplot as plt

from catboost import CatBoostClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

```

```

df = pd.read_csv("pca_transformed_dataset.csv")

print("Unique target values:", df.iloc[:, -1].unique())

X = df.iloc[:, :-1] # Features

y = df.iloc[:, -1] # Multi-class target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

catboost_clf = CatBoostClassifier(

    iterations=1000,

    depth=6,

    learning_rate=0.1,

    loss_function='MultiClass', # Enable multi-class classification

    verbose=200)

start_time = time.time()

catboost_clf.fit(X_train, y_train)

training_time = time.time() - start_time

y_pred = catboost_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\n CatBoost Multi-Class Classification Results:")

print(f"Accuracy: {accuracy:.2f}")

print("Classification Report:\n", classification_report(y_test, y_pred))

plt.figure(figsize=(8, 6))

cm = confusion_matrix(y_test, y_pred)

```

```

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=np.unique(y), yticklabels=np.unique(y))

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix - CatBoost Multi-Class")

plt.show()

```

3.3.3 KNN

```

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import time

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, roc_auc_score, recall_score, precision_score, f1_score,
cohen_kappa_score, matthews_corrcoef, confusion_matrix

from matplotlib.colors import ListedColormap

df = pd.read_csv("pca_transformed_dataset.csv")

X = df.iloc[:, :-1].values # All PCA components

y = df.iloc[:, -1].values # Target variable

```



```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)

start_time = time.time()

knn.fit(X_train, y_train)

training_time = time.time() - start_time

# Predictions

y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

auc = roc_auc_score(y_test, knn.predict_proba(X_test), multi_class="ovr") # FIXED HERE

recall = recall_score(y_test, y_pred, average="weighted") # Use weighted for multi-class

precision = precision_score(y_test, y_pred, average="weighted") # Use weighted for multi-class

f1 = f1_score(y_test, y_pred, average="weighted") # Use weighted for multi-class

kappa = cohen_kappa_score(y_test, y_pred)

mcc = matthews_corrcoef(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")

print(f"AUC: {auc:.4f}") # FIXED

print(f"Recall: {recall:.4f}")

print(f"Precision: {precision:.4f}")

```

```

print(f"F1 Score: {f1:.4f}")

print(f"Kappa: {kappa:.4f}")

print(f"MCC: {mcc:.4f}")

print(f"Training Time: {training_time:.4f} seconds")

plt.figure(figsize=(6, 5))

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=np.unique(y), yticklabels=np.unique(y))

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()

```

3.3.4 SVM

```

import numpy as np

import pandas as pd

from sklearn import svm

import seaborn as sns

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    matthews_corrcoef, roc_auc_score, confusion_matrix)

from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt

```

```

data = pd.read_csv("pca_transformed_dataset.csv")

X = data.iloc[:, :-1].values

y = data.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

model = svm.SVC(probability=True)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

y_prob = model.predict_proba(X_test)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred, average='weighted')

recall = recall_score(y_test, y_pred, average='weighted')

f1 = f1_score(y_test, y_pred, average='weighted')

mcc = matthews_corrcoef(y_test, y_pred)

auc = roc_auc_score(y_test, y_prob, multi_class='ovr', average='weighted')

cm = confusion_matrix(y_test, y_pred)

print(f'Accuracy: {accuracy:.4f}')

print(f'Precision: {precision:.4f}')

print(f'Recall: {recall:.4f}')

print(f'F1-Score: {f1:.4f}')

print(f'MCC: {mcc:.4f}')

print(f'AUC (OVR weighted): {auc:.4f}')

```

```

print('Confusion Matrix:')

print(cm)

classes = np.unique(y_test)

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

             xticklabels=classes, yticklabels=classes)

plt.title('Confusion Matrix Heatmap')

plt.xlabel('Predicted label')

plt.ylabel('True label')

plt.tight_layout()

plt.show()

```

3.3.5 QDA

```

import pandas as pd

import numpy as np

from sklearn.preprocessing import LabelEncoder, MinMaxScaler, label_binarize

from sklearn.decomposition import PCA

from sklearn.model_selection import train_test_split

from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

from sklearn.calibration import CalibratedClassifierCV, calibration_curve

```

```

from sklearn.metrics import (

    accuracy_score, precision_score, recall_score,

    f1_score, cohen_kappa_score, roc_auc_score,

    matthews_corrcoef, confusion_matrix)

import matplotlib.pyplot as plt

import seaborn as sns

df = pd.read_csv(r"C:\Users\aruld\Downloads\pca_transformed_dataset (4).xls")

for col in df.select_dtypes(include='object').columns:

    df[col] = LabelEncoder().fit_transform(df[col])

X = df.drop('attack_cat', axis=1)

y = df['attack_cat']

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)

pca_2d = PCA(n_components=2)

X_pca_2d = pca_2d.fit_transform(X_scaled)

X_pca_2d_df = pd.DataFrame(X_pca_2d, columns=['Feature One', 'Feature Two'])

X_train_2d, X_test_2d, y_train_2d, y_test_2d = train_test_split(

    X_pca_2d_df, y, test_size=0.25, stratify=y, random_state=42)

pca_full = PCA(n_components=X.shape[1])

X_pca_full = pca_full.fit_transform(X_scaled)

X_train, X_test, y_train, y_test = train_test_split(

```

```

X_pca_full, y, test_size=0.25, stratify=y, random_state=42

)qda = Quadratic Discriminant Analysis()

calibrated_qda = CalibratedClassifierCV(qda, cv=5, method='isotonic')

calibrated_qda.fit(X_train, y_train)

y_pred = calibrated_qda.predict(X_test)

y_proba = calibrated_qda.predict_proba(X_test)

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred, average='weighted')

recall = recall_score(y_test, y_pred, average='weighted')

f1 = f1_score(y_test, y_pred, average='weighted')

kappa = cohen_kappa_score(y_test, y_pred)

mcc = matthews_corrcoef(y_test, y_pred)

auc = roc_auc_score(y_test, y_proba, multi_class='ovr')

print(f'Accuracy:      {accuracy:.4f}')

print(f'Precision:     {precision:.4f}')

print(f'Recall:       {recall:.4f}')

print(f'F1 Score:      {f1:.4f}')

print(f'Cohen's Kappa: {kappa:.4f}')

print(f'AUC Score:     {auc:.4f}')

print(f'MCC:         {mcc:.4f}')

plt.figure(figsize=(6, 5))

```

```

sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues',

            xticklabels=np.unique(y), yticklabels=np.unique(y))

plt.title('QDA Confusion Matrix')

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.tight_layout()

plt.show()

classes = np.unique(y)

target_class = 0 # Change this to other class index if needed

y_test_bin = label_binarize(y_test, classes=classes)[: , target_class]

y_proba_class = y_proba[: , target_class]

fraction_of_positives, mean_predicted_value = calibration_curve(

    y_test_bin, y_proba_class, n_bins=10, strategy='uniform')

plt.figure(figsize=(10, 5))

plt.plot(mean_predicted_value, fraction_of_positives, "s-", label="QuadraticDiscriminantAnalysis",

        color="blue", linewidth=2, markersize=6)

plt.plot([0, 1], [0, 1], "k:", label="Perfectly calibrated", linewidth=1.5)

plt.title("Calibration plots (reliability curve)", fontsize=14)

plt.xlabel("Mean predicted probability", fontsize=12)

plt.ylabel("Fraction of positives", fontsize=12)

plt.legend(loc="lower right")

```

```

plt.grid(False)

plt.tight_layout()

plt.show()

qda_boundary = QuadraticDiscriminantAnalysis()

qda_boundary.fit(X_train_2d, y_train_2d)

x_min, x_max = X_train_2d.iloc[:, 0].min() - 1, X_train_2d.iloc[:, 0].max() + 1

y_min, y_max = X_train_2d.iloc[:, 1].min() - 1, X_train_2d.iloc[:, 1].max() + 1

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),
                     np.linspace(y_min, y_max, 500))

grid = np.c_[xx.ravel(), yy.ravel()]

Z = qda_boundary.predict(grid).reshape(xx.shape)

plt.figure(figsize=(10, 6))

plt.contourf(xx, yy, Z, alpha=0.5, cmap=plt.cm.coolwarm)

scatter = plt.scatter(X_train_2d.iloc[:, 0], X_train_2d.iloc[:, 1],
                     c=y_train_2d, edgecolor='k', cmap=plt.cm.coolwarm, s=50)

plt.xlabel('Feature One')

plt.ylabel('Feature Two')

plt.title('Decision Boundary of QDA')

plt.legend(*scatter.legend_elements(), title="Classes")

plt.tight_layout()

plt.show()

```


3.3.6 NAIVE BAYES

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, MinMaxScaler, label_binarize

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import (

    accuracy_score, precision_score, recall_score, f1_score,

    cohen_kappa_score, matthews_corrcoef, roc_auc_score,

    confusion_matrix, ConfusionMatrixDisplay)

from sklearn.calibration import calibration_curve

from sklearn.decomposition import PCA

data = pd.read_csv("processed_dataset.csv")

X = data.drop("attack_cat", axis=1)

y = data["attack_cat"]

le = LabelEncoder()

y_encoded = le.fit_transform(y)

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)
```

```

X_train, X_test, y_train, y_test = train_test_split(

    X_scaled, y_encoded, test_size=0.25, random_state=42, stratify=y_encoded

)nb = GaussianNB()

nb.fit(X_train, y_train)

y_pred = nb.predict(X_test)

y_proba = nb.predict_proba(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Precision:", precision_score(y_test, y_pred, average='weighted', zero_division=0))

print("Recall:", recall_score(y_test, y_pred, average='weighted', zero_division=0))

print("F1 Score:", f1_score(y_test, y_pred, average='weighted', zero_division=0))

print("Kappa:", cohen_kappa_score(y_test, y_pred))

print("MCC:", matthews_corrcoef(y_test, y_pred))

print("AUC:", roc_auc_score(y_test, y_proba, multi_class='ovr'))

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)

disp.plot(cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix")

plt.show()

y_test_binarized = label_binarize(y_test, classes=np.unique(y_encoded))

n_classes = y_test_binarized.shape[1]

y_true_flat = y_test_binarized.ravel()

```

```

y_proba_flat = y_proba.ravel()

prob_true, prob_pred = calibration_curve(y_true_flat, y_proba_flat, n_bins=10)

plt.figure(figsize=(8, 5))

plt.plot(prob_pred, prob_true, marker='s', linestyle='-', label="GaussianNB")

plt.plot([0, 1], [0, 1], linestyle='--', color='black', label="Perfectly calibrated")

plt.xlabel("Mean Predicted Probability")

plt.ylabel("Fraction of Positives")

plt.title("Calibration Plot (GaussianNB - All Classes)")

plt.legend()

plt.grid()

plt.show()

pca = PCA(n_components=2)

X_pca_2d = pca.fit_transform(X_scaled)

X_pca_train, X_pca_test, y_pca_train, y_pca_test = train_test_split(

    X_pca_2d, y_encoded, test_size=0.25, random_state=42, stratify=y_encoded)

nb_2d = GaussianNB()

nb_2d.fit(X_pca_train, y_pca_train)

x_min, x_max = X_pca_train[:, 0].min() - 1, X_pca_train[:, 0].max() + 1

y_min, y_max = X_pca_train[:, 1].min() - 1, X_pca_train[:, 1].max() + 1

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),

    np.linspace(y_min, y_max, 500))

```

```

grid = np.c_[xx.ravel(), yy.ravel()]

Z = nb_2d.predict(grid)

Z = Z.reshape(xx.shape)

plt.figure(figsize=(10, 6))

plt.contourf(xx, yy, Z, alpha=0.5, cmap=plt.cm.coolwarm)

for cls in np.unique(y_pca_train):

    idx = y_pca_train == cls

    marker = 's' if cls == 0 else 'o' # example: square for 0, circle for 1

    plt.scatter(X_pca_train[idx, 0], X_pca_train[idx, 1], label=str(cls),

                edgecolor='k', s=50, marker=marker, cmap=plt.cm.coolwarm)

plt.xlabel("Feature One")

plt.ylabel("Feature Two")

plt.title("Decision Boundary of NB")

plt.legend(title="Classes")

plt.tight_layout()

plt.show()

```

CHAPTER 4

RESULTS

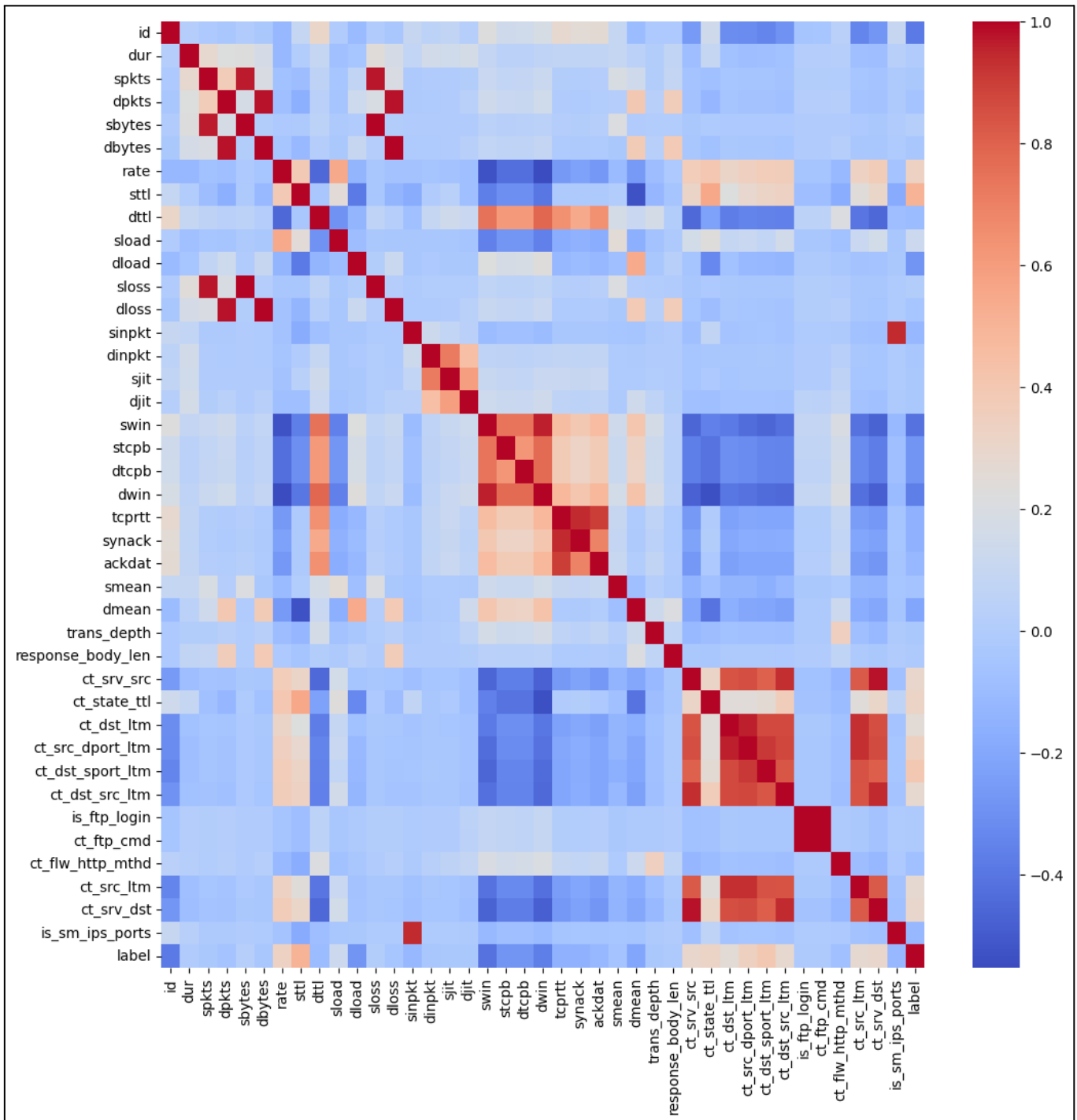


Fig 4.1 Heatmap obtained from EDA

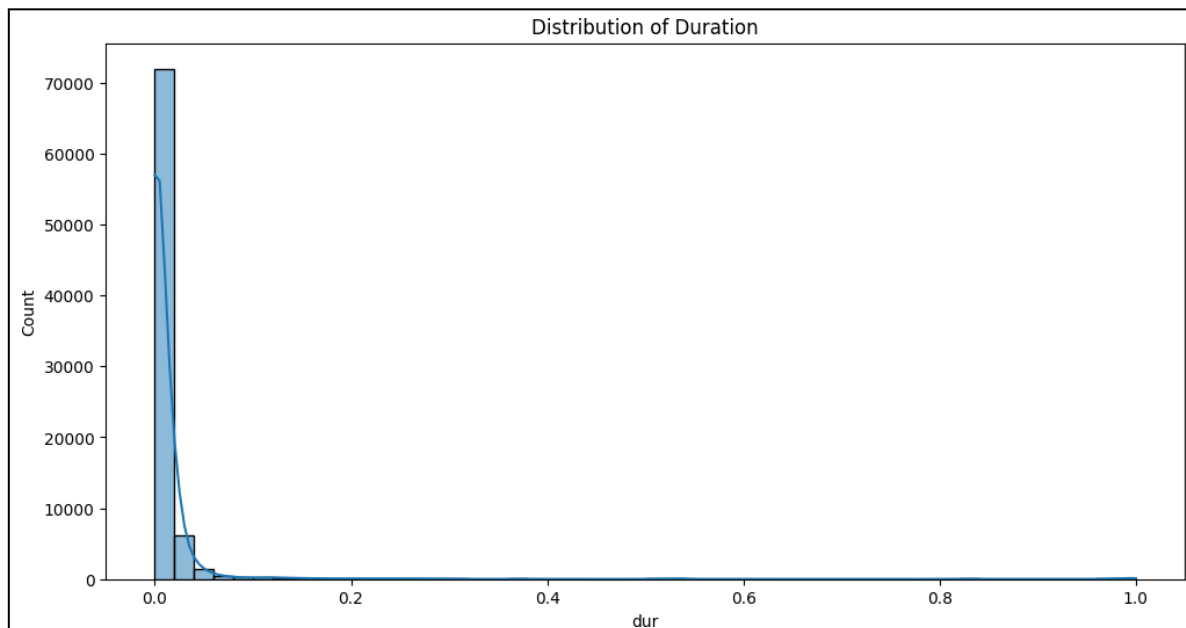


Fig 4.2 Distribution of duration

```
Label Distribution:
label
1    45332
0    37000
Name: count, dtype: int64

Attack Category Distribution:
attack_cat
0.666667    37000
0.555556    18871
0.333333    11132
0.444444     6062
0.222222     4089
0.777778     3496
0.000000       677
0.111111       583
0.888889       378
1.000000        44
Name: count, dtype: int64
```

Fig 4.3 Label Distribution

```

Class mappings: {'Analysis': 0, 'Backdoor': 1, 'DoS': 2, 'Exploits': 3, 'Fuzzers': 4, 'Generic': 5, 'Normal': 6, 'Reconnaissance': 7, 'Shellcode': 8, 'Worms': 9}
Dataset preprocessing complete! Saved as 'processed_dataset.csv'

```

Fig 4.4 Each Class mappings

```

Outlier Count per Feature:
[1063 2842 2123    6  218  701  102  649 1036    0    0 1372 2737  127
 653  912  437  579  610    0    0    0    0 638  544  633 2840 3001
   34  224 2235 1805 2644 2583 3195 2372  678  680  654 2408 2184  916
 677    0]

```

Fig 4.5 Outlier Count per Feature

```

Classification Report:
              precision    recall  f1-score   support

     0           0.00       0.00       0.00         163
     1           0.00       0.00       0.00         153
     2           0.45       0.74       0.56         985
     3           0.67       0.79       0.72        2828
     4           0.67       0.10       0.18        1515
     5           0.89       0.97       0.93        4695
     6           1.00       1.00       1.00        9251
     7           0.89       1.00       0.94         888
     8           0.00       0.00       0.00          95
     9           0.00       0.00       0.00          10

 accuracy              0.87        20583
 macro avg           0.46       0.46       0.43        20583
 weighted avg        0.85       0.87       0.84        20583

Accuracy: 0.8659087596560269

```

Fig 4.6 Classification report of PCA

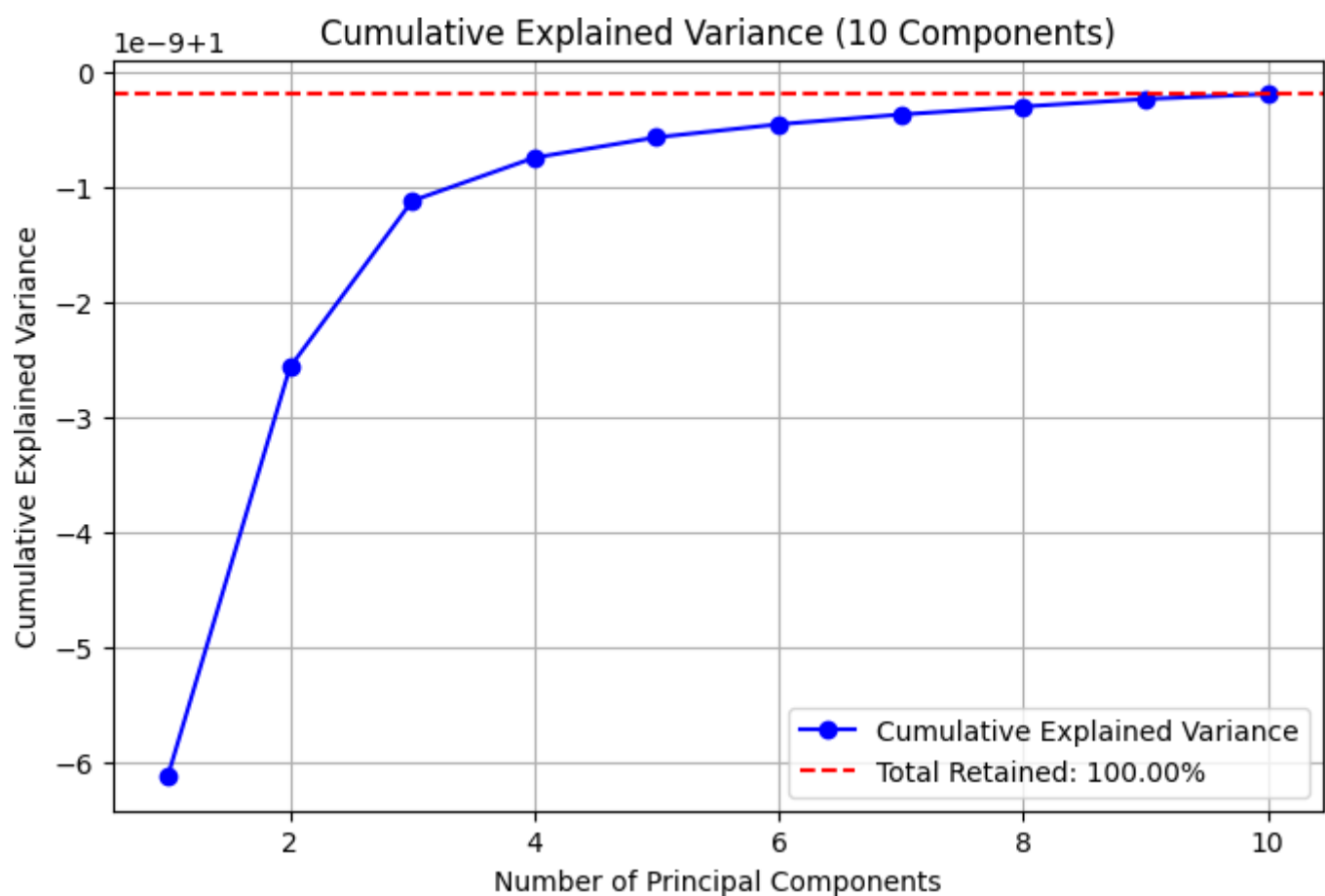


Fig 4.7 Cumulative Explained Variance graph of PCA

```

Accuracy: 98.15%
AUC: 99.13%
Recall: 83.33%
Precision: 87.64%
F1 Score: 85.19%
Kappa: 97.42%
MCC: 97.43%
Training Time: 8.9873 seconds

```

Fig 4.8 Performance metrics for XGBoost

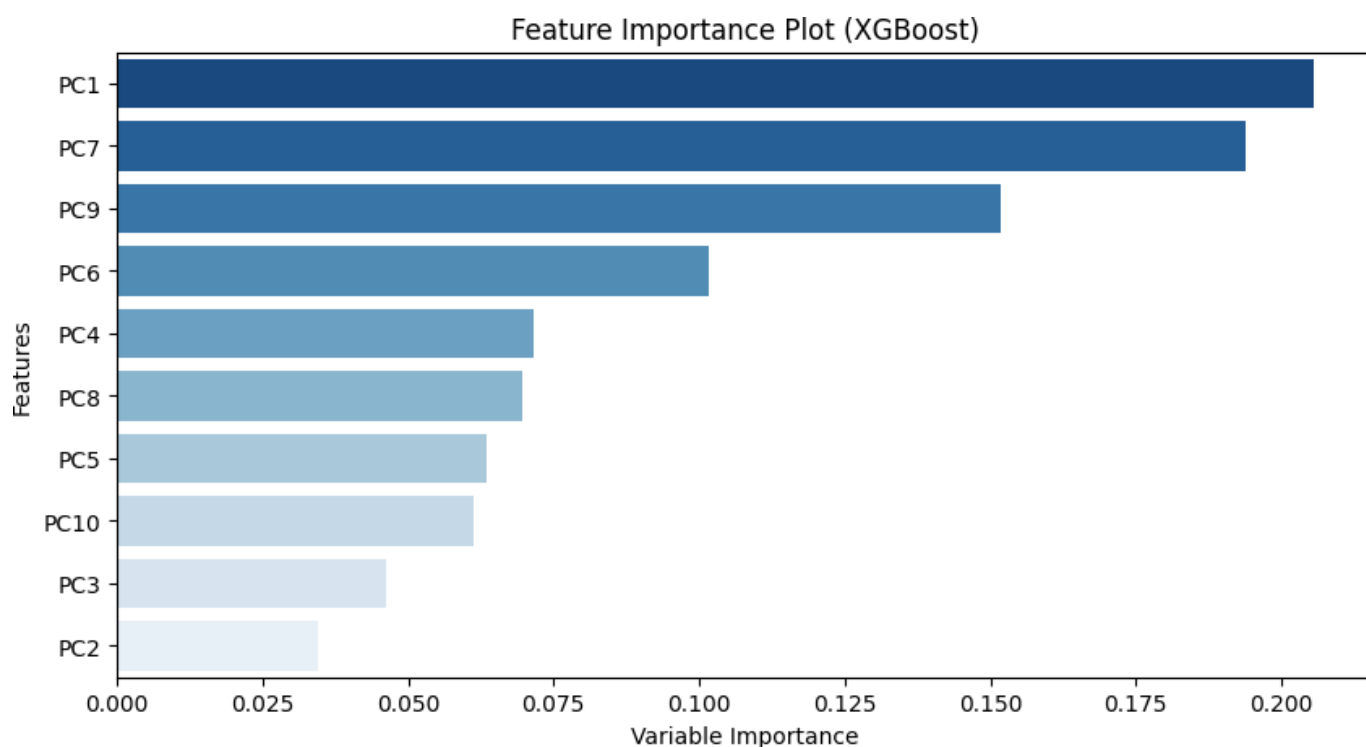


Fig 4.9 Feature importance plot of XGBOOST

```

Unique target values: [6 7 1 2 3 0 4 9 8 5]
0:      learn: 1.7179835      total: 357ms      remaining: 5m 56s
200:    learn: 0.0997947      total: 27.7s      remaining: 1m 50s
400:    learn: 0.0617505      total: 54.9s      remaining: 1m 21s
600:    learn: 0.0449140      total: 1m 22s      remaining: 54.5s
800:    learn: 0.0354897      total: 1m 49s      remaining: 27.1s
999:    learn: 0.0291882      total: 2m 16s      remaining: 0us

✅ CatBoost Multi-Class Classification Results:
Accuracy: 0.99
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	135
1	0.97	0.91	0.94	117
2	0.97	0.92	0.94	818
3	0.96	0.98	0.97	2227
4	0.97	0.97	0.97	1212
5	1.00	0.99	0.99	3774
6	1.00	1.00	1.00	7400
7	0.96	0.99	0.97	699
8	0.98	0.66	0.79	76
9	1.00	0.22	0.36	9
accuracy			0.99	16467
macro avg	0.98	0.86	0.89	16467
weighted avg	0.99	0.99	0.99	16467

Fig 4.10 Classification report and target values of CATBOOST

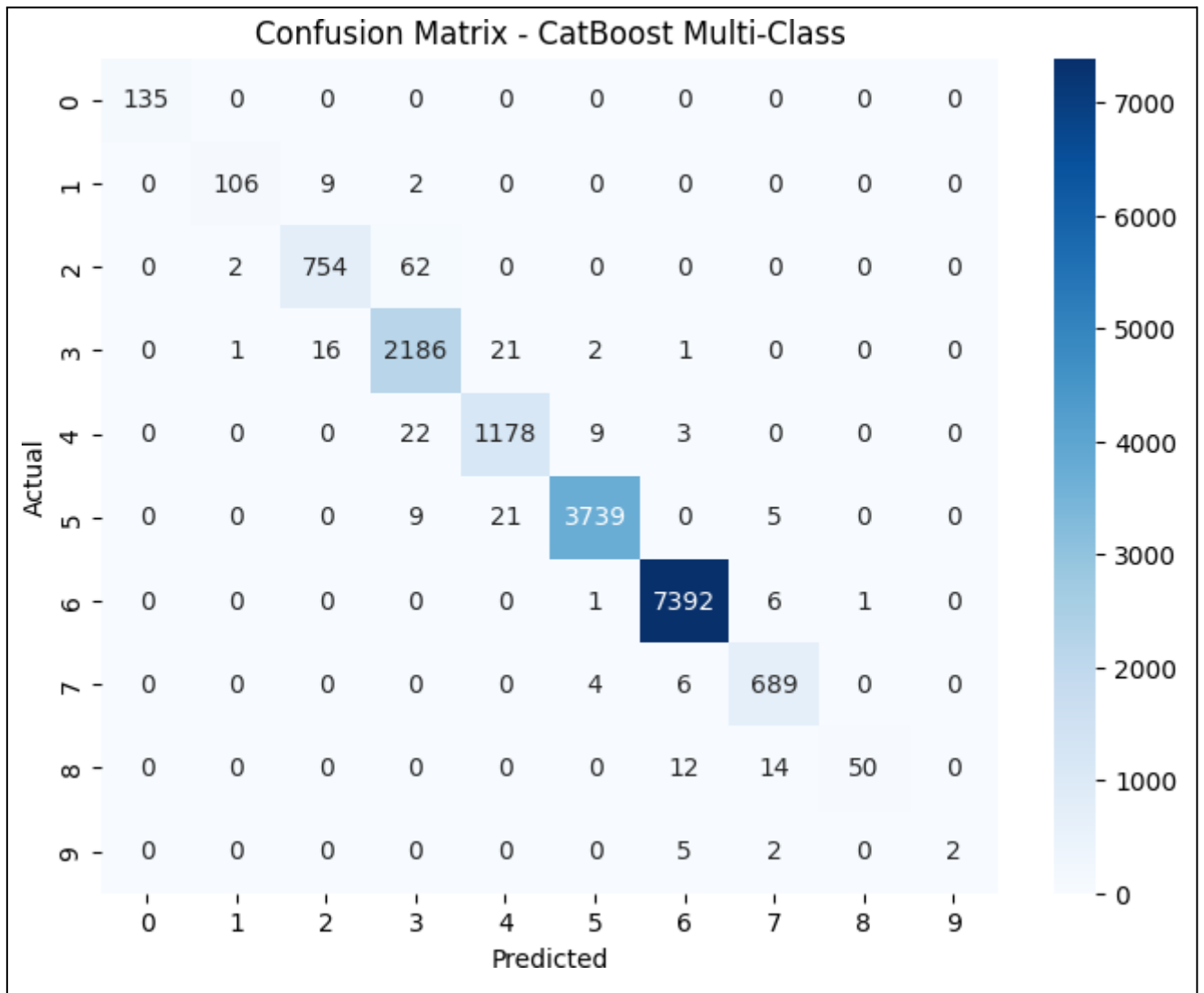


Fig 4.11 Confusion matrix of CATBOOST

Accuracy: 0.9842
AUC: 0.9717
Recall: 0.9842
Precision: 0.9843
F1 Score: 0.9839
Kappa: 0.9779
MCC: 0.9779
Training Time: 0.2352 seconds

Fig 4.12 Performance metrics for KNN

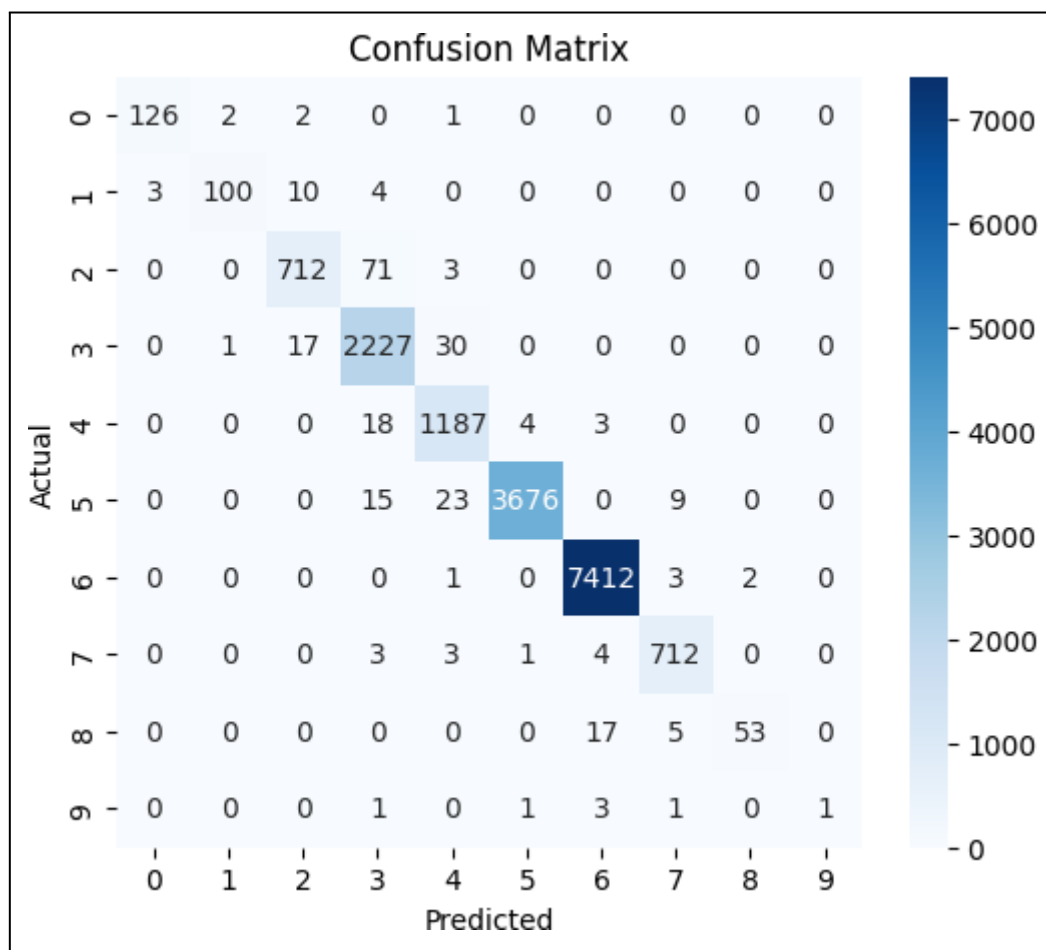


Fig 4.13 Confusion matrix for KNN

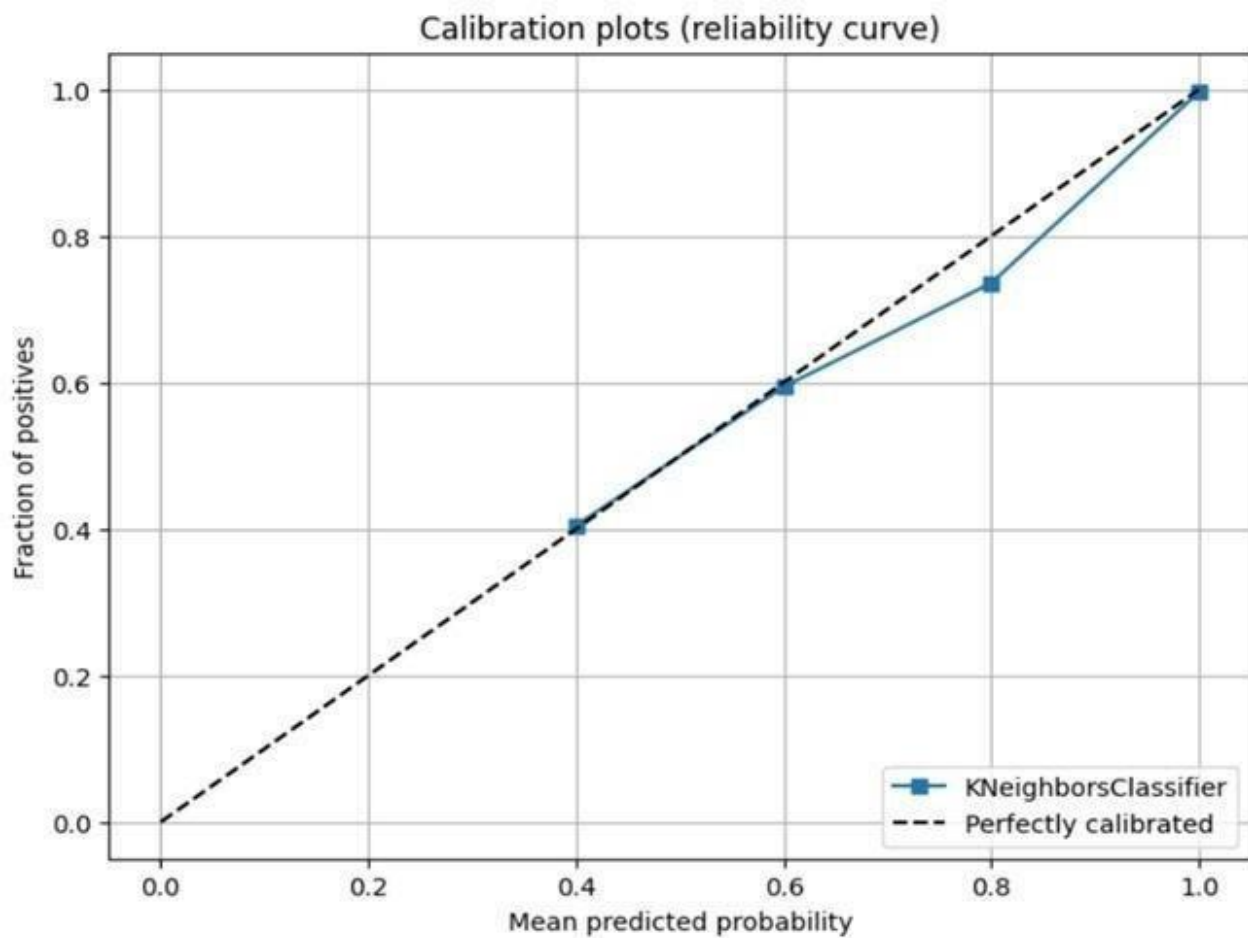


Fig 4.14 Calibration plot for KNN

```
Accuracy: 0.9624
Precision: 0.9598
Recall: 0.9624
F1-Score: 0.9573
MCC: 0.9476
AUC (OVR weighted): 0.9982
```

Fig 4.15 Performance metrics for SVM

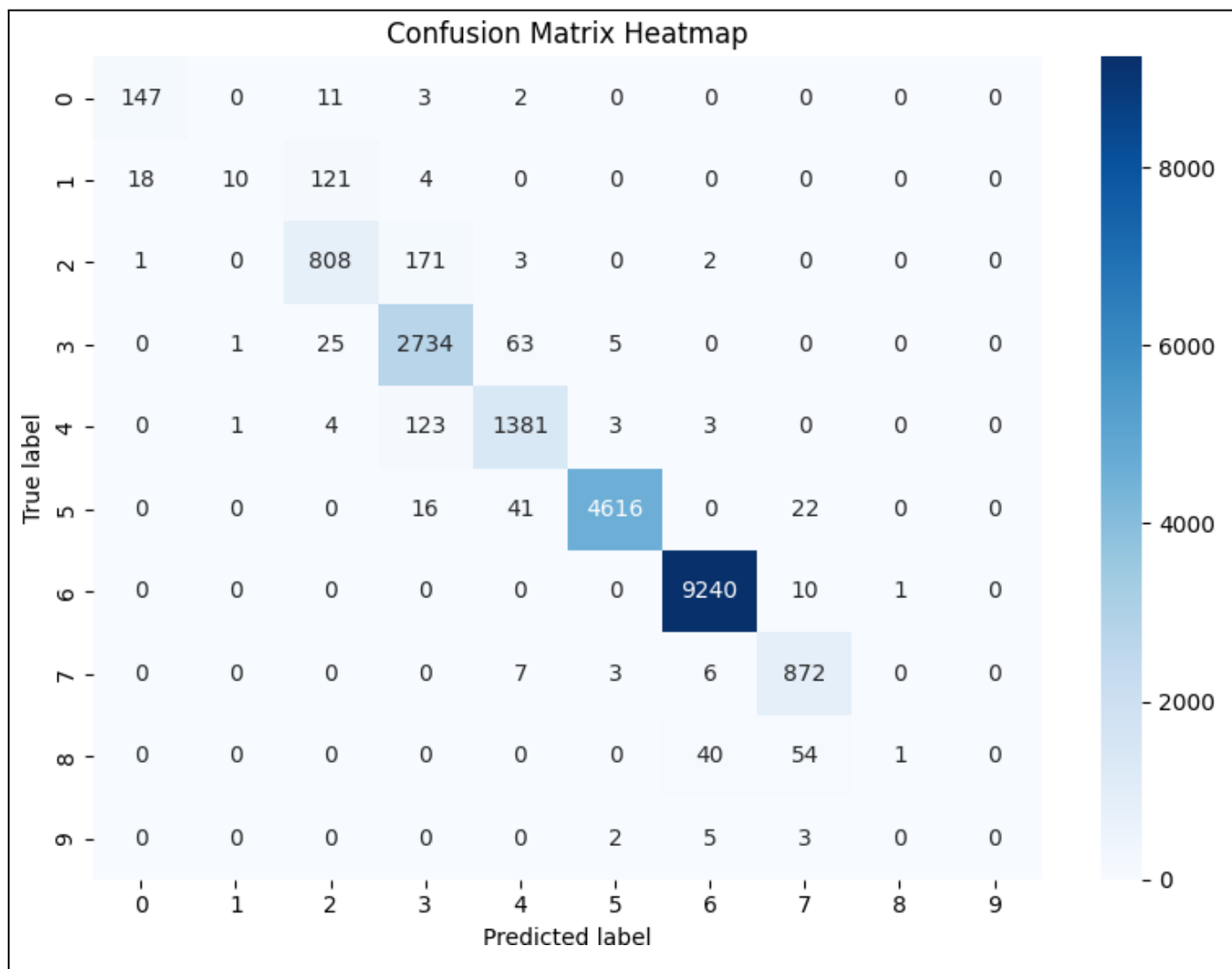


Fig 4.16 Confusion matrix for SVM

Accuracy:	0.9088
Precision:	0.9114
Recall:	0.9088
F1 Score:	0.9040
Cohen's Kappa:	0.8711
AUC Score:	0.9801
MCC:	0.8729

Fig 4.17 Performance metrics for QDA

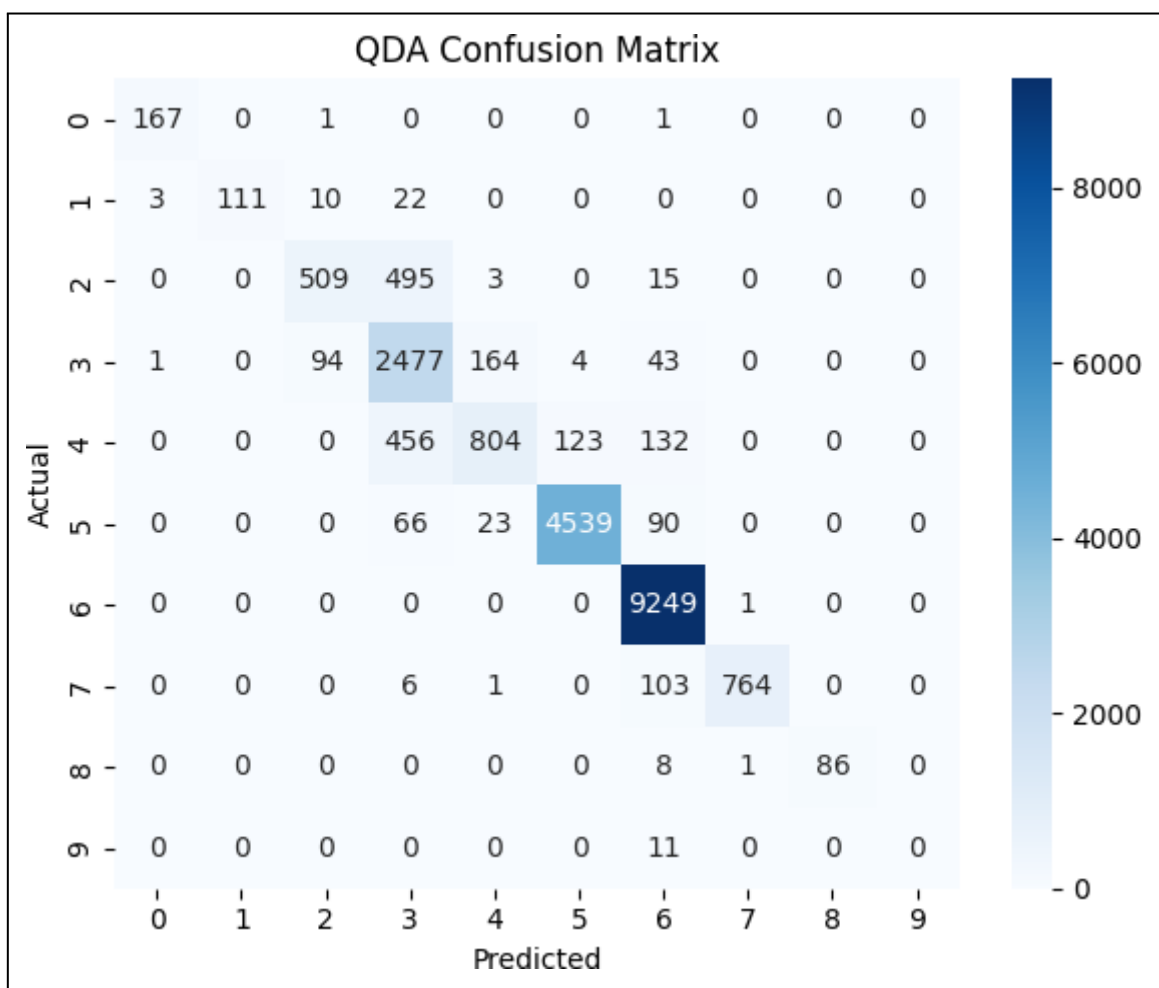


Fig 4.18 Confusion matrix for QDA

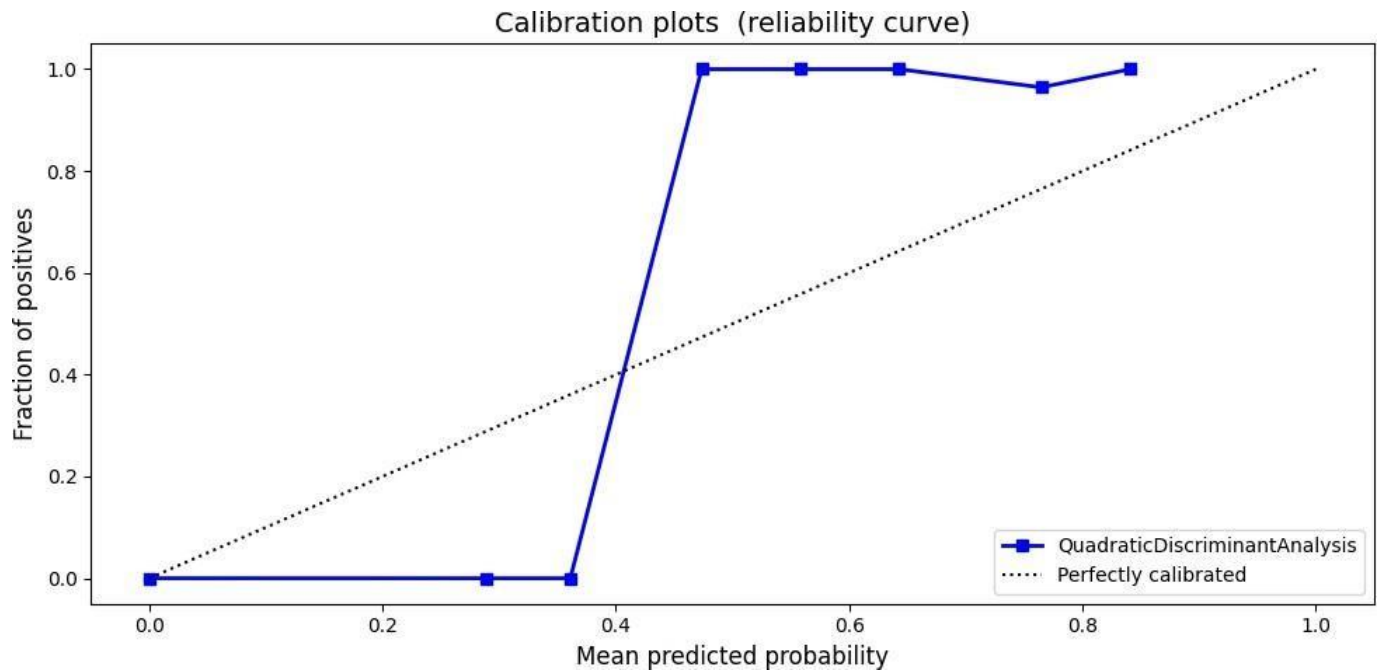


Fig 4.19 Calibration plot for QDA

```

Accuracy: 0.6717193800709323
Precision: 0.8679966253079222
Recall: 0.6717193800709323
F1 Score: 0.7227803905514296
Kappa: 0.5642191746703864
MCC: 0.5803114407250939
AUC: 0.9143421667644687

```

Fig 4.20 Performance metrics for Naive Bayes

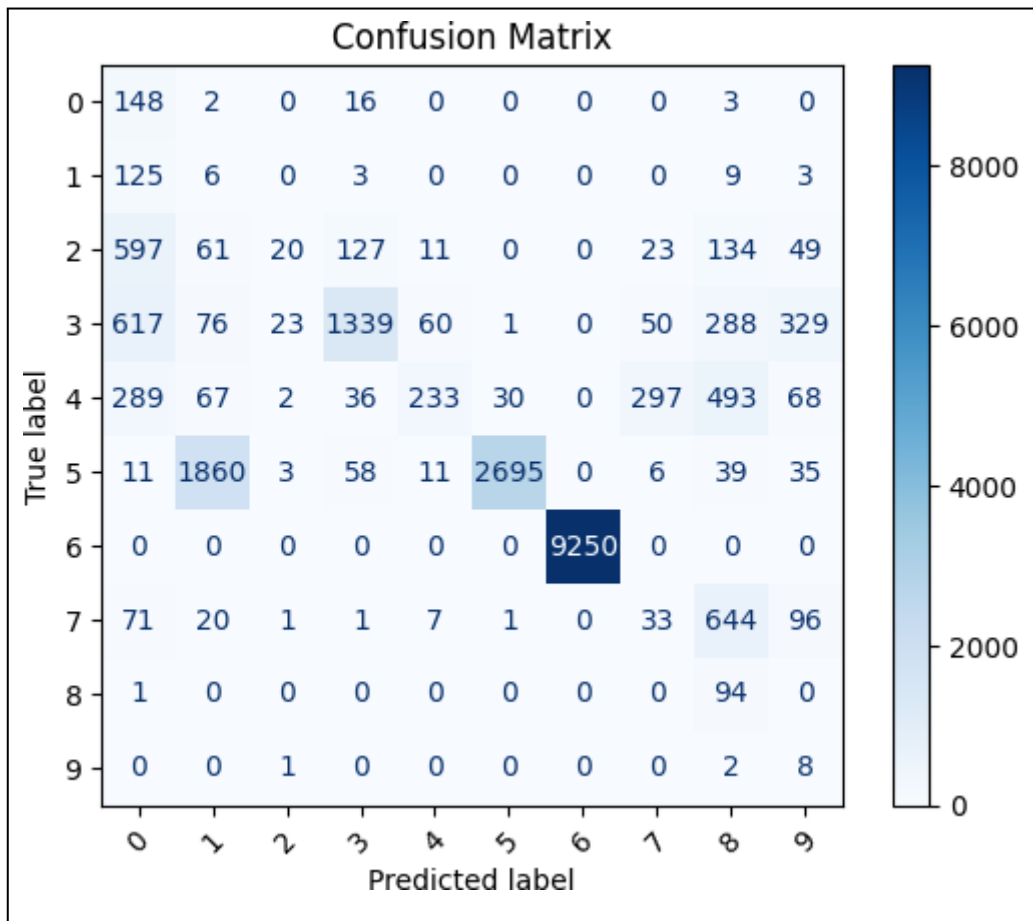


Fig 4.21 Confusion matrix for Naive Bayes

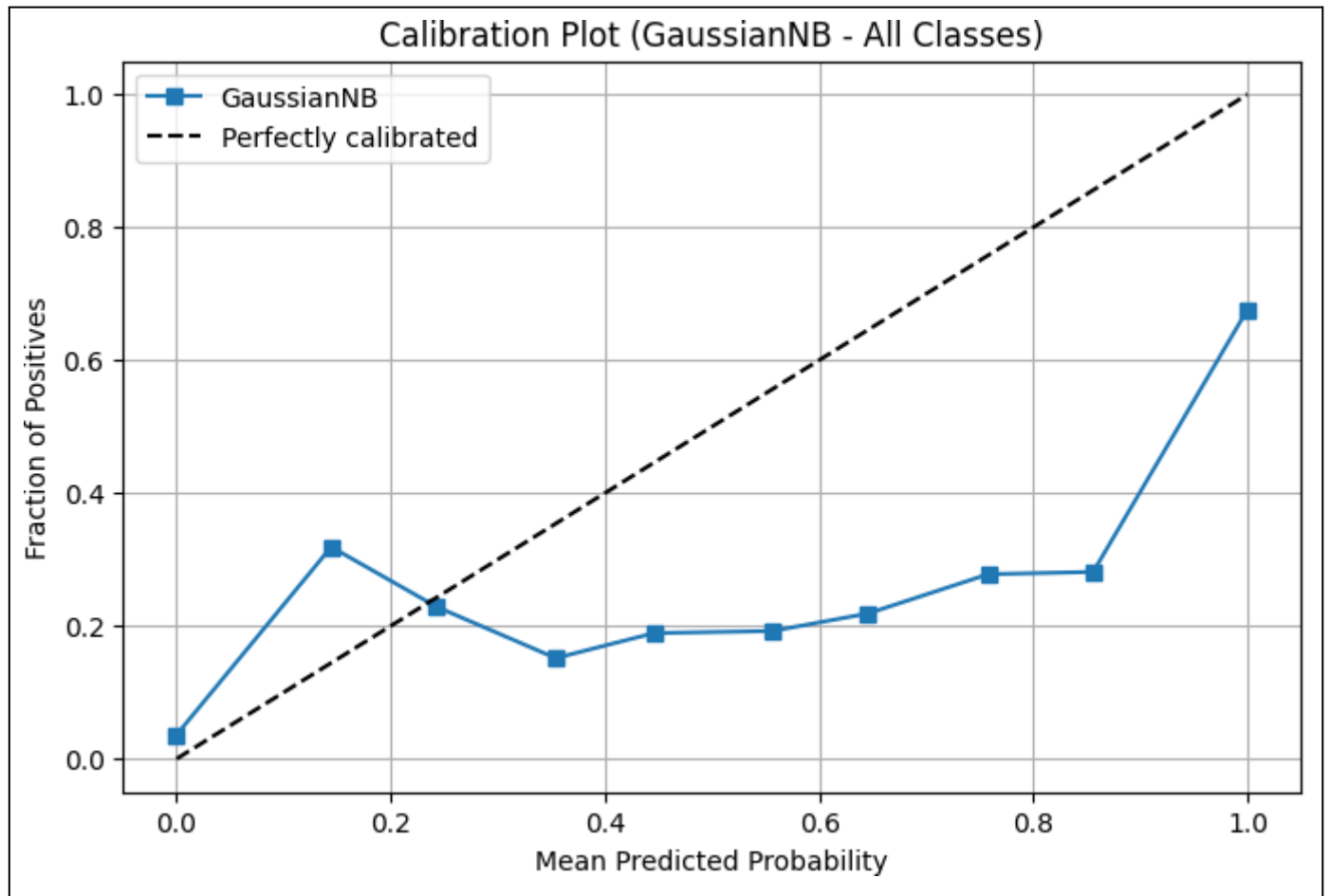


Fig 4.22 Calibration plot for Naive Bayes

CHAPTER 5

CONCLUSION AND FUTURE PLANS

The main objective of this paper was to develop a Machine Learning based Intrusion Detection system for detecting network attacks by implementation of Min-Max normalization, PCA, and classification algorithms like XGBoost, CatBoost, KNN, SVM, QDA and Naïve Bayes and the results performed high values especially with CatBoost and XGBoost by achieving 99% and 98.15% respectively.

Looking ahead, our primary goal is to boost the detection accuracy of the system while also refining key performance metrics like precision, recall, and the Matthews correlation coefficient (MCC). We aim to lower the overall error rate. A major part of the future work involves testing the proposed framework on larger, more diverse IoT datasets that include a broader range of attack types and real-world network conditions. This will allow us to better evaluate how well the model performs across different environments and ensure it remains effective and adaptable in practical applications.

CHAPTER 6

REFERENCES

- 1.Q. Niyaz, W. Sun, A.Y. Javaid, M. Alam, “A deep learning approach for network intrusion detection system”, EAI Int. Conf. Bio-inspired Inf. Commun. Technol. (2015)
- 2.E. Hodo et al., “Threat analysis of IoT networks using artificial neural network intrusion detection system,” 2016 Int. Symp. Networks, Comput. Commun. ISNCC 2016, pp. 4–9, 2016
- 3.H. Bostani, M. Sheikhan, Hybrid of anomaly-based and specification-based IDS for Internet of Things using unsupervised OPF based on MapReduce approach, *Comput. Commun.* 98 (2017) 52–71
- 4.H.H. Pajouh, R. Javidan, R. Khayami, A. Dehghantanha, K. K. Choo, A Two-Layer Dimension Reduction and Two-Tier Classification Model for Anomaly-Based Intrusion Detection in IoT Backbone Networks, *IEEE Trans. Emerg. Top. Comput.*
5. A.A. Diro, N. Chilamkurti, Distributed Attack Detection Scheme using Deep Learning Approach for Internet of Things, *Futur. Gener. Comput. Syst.* 82 (2018) 761–768.
6. E. Anthi, L. Williams, P. Burnap, Pulse: An adaptive intrusion detection for the internet of things, *IET Conf. Publ.* 2018 (CP740) (2018).
7. R. Kozik, M. Choras, M. Ficco, F. Palmieri, A scalable distributed machine learning approach for attack detection in edge computing environments, *J. Parallel Distrib. Comput.* 119 (2018)
- 8.G. Thamilarasu and S. Chawla, “Towards deep-learning-driven intrusion detection for the internet of things,” *Sensors* (Switzerland), vol. 19, no. 9, 2019,
- 9.S.U. Jan, S. Ahmed, V. Shakhov, I. Koo, Toward a Lightweight Intrusion Detection System for the Internet of Things, *IEEE Access* 7 (2019) 42450–42471.

CHAPTER-7

APPENDIX - BASE PAPER

Base Paper : Yakub Kayode Saheed, Aremu Idris Abiodun, Sanjay Misra, Monica Kristiansen Holone, Ricardo Colomo-Palacios, “A Machine Learning-Based Intrusion Detection for Detecting Internet of Things Network Attacks,” *Alexandria Engineering Journal*, vol. 61, pp. 9395–9409, 2022.

Doi : 10.1016/j.aej.2022.02.063

keywords : Internet of Things, Intrusion Detection System, Machine Learning, Support Vector Machine, K-Nearest Neighbour, Naïve Bayes, Principal Component Analysis.

URL : <https://doi.org/10.1016/j.aej.2022.02.063>